

Step 1 — Create files in .helix/

Validate the PR first

Before touching Docker or writing any files, review the `existing-solution` PR to confirm the task is worth working on.

Do not skip this step

Many fellows jump straight into Docker setup without validating the PR. If the PR turns out to be invalid, all that work is wasted. Check the PR now — before you write a single file.

- Open the existing-solution PR¹ and review the diff.
- Count the lines of real logic changed² — ignore lock files, generated code, and formatting-only changes.
- Decide³: if the logic changes are **fewer than 25 LOC** or **more than 500 LOC**, **flag the task** and do not proceed. Otherwise, continue to Docker setup below.

1. Navigate to the PR from the task page

2. Only meaningful code changes count

3. Flagging frees the task for reassignment

Flag, don't force it

If the PR is outside the 25–500 LOC range, flag it immediately. Do not proceed with Docker setup or golden-solution work on a flagged task.

Create files in .helix/

The goal here is to validate that we're able to successfully run the tests in this repository. In order to reduce variations based on operating system, library versions, etc, we're using Docker to ensure a consistent environment.

Your job is to add three files inside the `.helix/` folder that let us create a stable environment to run tests.

Dockerfile.helix

A Dockerfile that pins the base language of the repo at the point in time of the PR, along with any additional dependencies required to run the test suite (e.g. `pytest`, `jest`, `vitest`).

GIT — MANDATORY

The Dockerfile MUST install git as a system dependency (e.g. `apt-get install -y git`). Many build tools, package managers, and version plugins require git to be present.

SSL CERTIFICATES — MANDATORY

The Dockerfile MUST also install SSL certificates. If you are running a Debian/Ubuntu based image, add ca-certificates to the apt-get install line (e.g. `RUN apt-get update && apt-get install -y ca-certificates`). If using an Alpine based image (anything with a '-slim' suffix), use `RUN apk add --no-cache ca-certificates` instead.

Go repos: use exec-form CMD/ENTRYPOINT

For Go-based task repos, the Dockerfile must use exec-form for CMD or ENTRYPOINT — e.g. `CMD ["go", "test", "./..."]`. Do NOT use shell-form like `CMD ["sh", "-c", "go list ./... | ..."]`. The evaluation runner detects the test runner from exec-form arguments; shell-form falls back to 'unknown' and runs the full test suite instead of the targeted paths the evaluator requested. This silently breaks Pass@k yield on Go tasks.

run-tests-eval.sh

A Bash script that runs the repo's test suite. It must handle two modes:

- No arguments¹ — run **all** tests in the repo.
- With arguments² — accept comma-separated file paths and run only those tests.

- ¹. Default mode for full validation
- ². Used by the evaluation harness for targeted runs

The script runs tests **directly** (e.g. `pytest`, `jest`, `vitest`) — it must **not** invoke Docker itself.

- Example: `joke2k_faker_2264 /.helix/run-tests-eval.sh`

No Docker inside the script

`run-tests-eval.sh` should call the test runner directly (e.g. `pytest tests/`). The CI workflow handles Docker — your script just runs tests.

If the repo has many flaky tests, you may prune them so that every test that runs also passes.

metadata.json

A placeholder metadata file that will be filled in during Step 6 of the Core Workflow. For now, create the file with the initial structure specified in the prompt below.

Only required files

The `.helix/` folder must contain only the three required files: `Dockerfile.helix`, `run-tests-eval.sh`, and `metadata.json`. Do not add any extra files or modify other files in the repo. Remove any additional files from `.helix/` before submitting.

Starter prompt for Cursor

We recommend using an LLM to help write the script. Here's a starter prompt you can adapt:

I need to create the `.helix/` folder for this repo with three files:



1. `Dockerfile.helix`

- Use a base image matching the repo's language and version at the current commit.
- Install all dependencies needed to run the test suite (e.g. `pytest`, `jest`, `vitest`).
- MANDATORY: Install `git` as a system dependency (e.g. `apt-get install -y git`). Many build tools, package managers, and version plugins require `git` to be present.
- SSL CERTIFICATES – MANDATORY:
 - The Dockerfile MUST install SSL certificates (`ca-certificates` package).
 - For Debian/Ubuntu images: add `ca-certificates` to the `apt-get install` line.
 - For Alpine images (anything with a `-slim` suffix): use `RUN apk add --no-cache ca-certificates`.
- Copy the repo contents into the image.

2. `run-tests-eval.sh`

- When called with NO arguments, run ALL tests in the repo.
- When called WITH comma-separated file paths, run ONLY those test files.
- Call the test runner directly (e.g. `pytest`, `jest`) – do NOT use docker commands.
- The script must be executable (`chmod +x`).

3. `metadata.json`

- Create a placeholder JSON file with this exact initial structure:

```
[{"instance_id":"handshake","task_title":"handshake","problem_statement":"","hints":
```

Look at the repo's existing test setup and dependencies to determine the correct base image, dependencies, and test command.

Reference

- **Example repo (no `.helix` yet):** [Textualize rich 3942](#) — root has no `.helix`; `.github/workflows/helix-validation.yml` is present.
- **Example `.helix` files:** [joke2k faker 2264 /.helix](#).